

EC 504
Spring, 2025
Exam 1

Take Home: Due Monday, March 24, 2025

Last Name	First Name	Student ID #
-----------	------------	--------------

Honor Code: This exam represents only my own work. I did not give or receive help.

Signature: _____

Instructions

- Print your name and BU ID on the first page carefully so automatic text recognition will identify you.
- This exam allows you to use 3 double sided pages of notes. No consultation with anyone other than the instructors are permitted during the exam.
- You have one hour and 50 minutes to complete this exam. It is designed to require less than one hour and fifteen minutes of work, so there should not be significant time pressure. Nevertheless, there are many problems, so do not spend too much time on any one question; you can always return to it.

Problem 1 (12 pts)

Answer True or False to each of the questions below. Each question is worth 2 points. Answer true only if it is true as stated, with no additional assumptions. No explanation is needed, but any rational explanation is likely to earn partial credit, and no explanation will not earn any credit if the answer is wrong.

(a) $n^2 \log(n) \in O(n^{2.1})$

Solution:

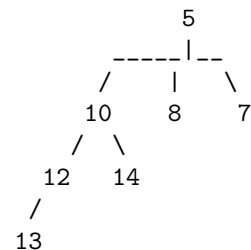
True. Note that $n^{0.1} > \log(n)$ as n increases!

(b) We have a binary min-heap with n elements and wish to insert an array with n more elements into the heap (not necessarily one at a time) to form a min-heap with $2n$ elements. The total time required for this is $\Theta(n \log(n))$.

Solution:

False. Just put the two arrays back to back, and build the heap. $\Theta(n)$.

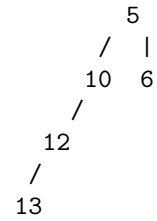
(c) The tree on the right is an example of a tree that can occur in a binomial min-heap:



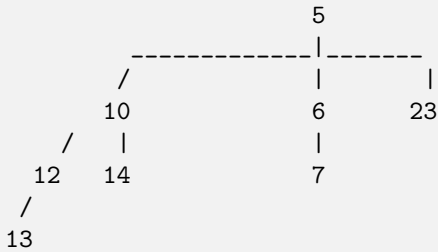
Solution:

False. it is not a binomial tree because it has two children of order 0, no child of order 1.

- (d) The tree on the right is an example of a tree that can occur in a Fibonacci min-heap by reducing keys in a binomial tree of order 3.

**Solution:**

True. Start from a binomial tree of order 3 as shown below:



Now, reduce keys of 23 to 0, 7 to 1, 14 to 2.

- (e) Consider a B+-tree in which the maximum number of keys in a non-leaf node is 9. Then, the minimum number of keys in any non-root, non-leaf node is 5.

Solution:

False. Max keys = 9 means 10 children, so minimum number of children is 5, which corresponds to 4 keys.

- (f) In the KMP algorithm for string matching, the prefix function for the string “AAABBBBA” is [0,1,2,0,1,2,1].

Solution:

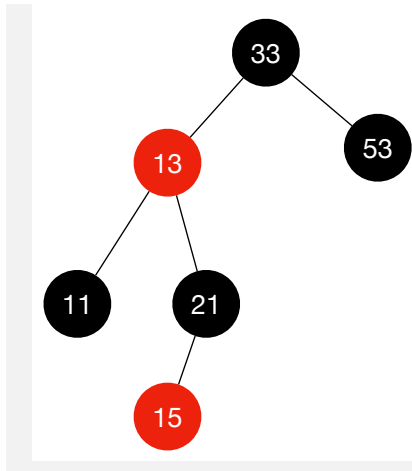
False. It is [0,1,2,0,0,0,1].

Problem 2 (16 pts) Quick questions

- (a) The height of a tree is the length of the longest root-to-leaf path in it, in terms of number of edges in the path. What is the minimum number of nodes in a red-black tree of height 3?

Solution:

The minimum number of nodes is 6, illustrated by this tree with black height 2.



- (b) Consider a complete binary tree with n elements, stored in an array, where the left and the right subtrees of the root are ordered as **max-heaps**. Describe an algorithm with worst case complexity $\Theta(n)$ for converting this tree to a **min-heap**.

Solution:

This is what you get when you do build a heap from any array of n elements using heapify, which is $\Theta(n)$.

- (c) Consider two binomial heaps H_1 , H_2 , where H_1 has binomial trees of rank 0, 1, 3 and 4, and H_2 has binomial trees of ranks 0, 3, and 4. What are the ranks of the binomial trees that result when we join (merge, not lazy merge) H_1 and H_2 ?

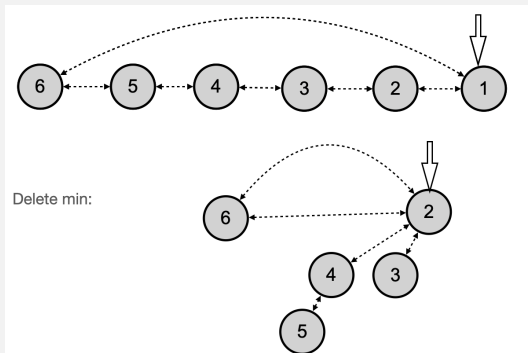
Solution:

The merged $H_1 + H_2$ will have trees of ranks 2, 4, 5. The two rank 0 trees will merge into a rank 1 tree, which merges with the rank 1 tree to form a rank 2 tree. The two rank 3 trees will merge into a rank 4 tree, creating 3 rank 4 trees. Two of the rank 4 trees will merge into a rank 5 tree.

- (d) Insert the following keys into an empty Fibonacci heap, then delete the minimum key and show the final Fibonacci heap that results: 1,2,3,4,5,6.

Solution:

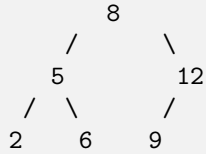
See figure below.



Problem 3 (6 pts)

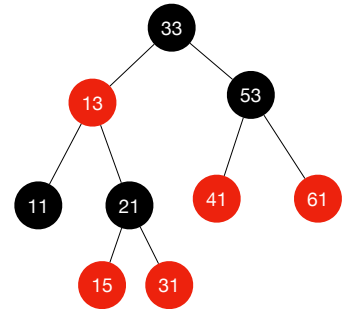
Given a binary search tree, **pre-order** traversal of this tree yields the following sequence: 8, 5, 2, 6, 12, 9. Draw a complete binary search tree of height 2 consistent with this traversal.

Solution:



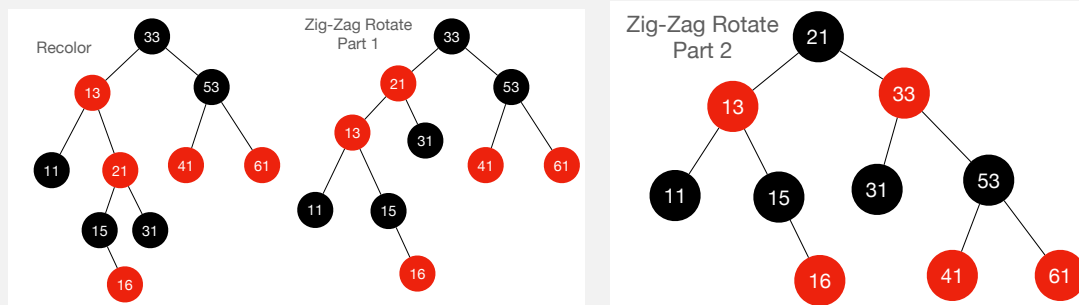
Problem 4 (6 pts)

Consider the red-black tree shown on the right. Show the red-black tree that results when you insert the key 16 into the tree and restore the red-black property.



Solution:

The solution is illustrated below. It involves a recolor, then a zig-zag rotation.



Problem 5 (6 pts)

We are going to use cuckoo hashing with two hash tables of length 5 (so positions 0,1,2,3,4 in each), with hash functions $h_1(k) = k \bmod 5$ for the first table, and hash function $h_2(k) = \lfloor \frac{k}{5} \rfloor \bmod 5$ for the second table. Insert the following keys into the hash tables using cuckoo hashing in this order: 12, 22, 2, 52, 32. Fill the numbers in the table below, where each row represents the stored elements at the end of each insert:

Table 1 slots:	0	1	2	3	4	Table 2:	0	1	2	3	4
Insert 12											
insert 22											
insert 2											
insert 52											

Solution:

If you do the first 4 inserts, you get this:

Table 1 slots:	0	1	2	3	4	Table 2:	0	1	2	3	4
Insert 12			12								
insert 22			22						12		
insert 2			2						12		22
insert 52			52				2		12		22

If you do all 5, you get this.

Table 1 slots:	0	1	2	3	4	Table 2:	0	1	2	3	4
Insert 12			12								
insert 22			22						12		
insert 2			2						12		22
insert 52			52				2		12		22
insert 32			2				52	32	12		22

Problem 6 (6 pts)

A binary min-heap implemented using an array is given as [8, 10, 16, 19, 18, 17, 20]. What is the content of the array after two delete min operations?

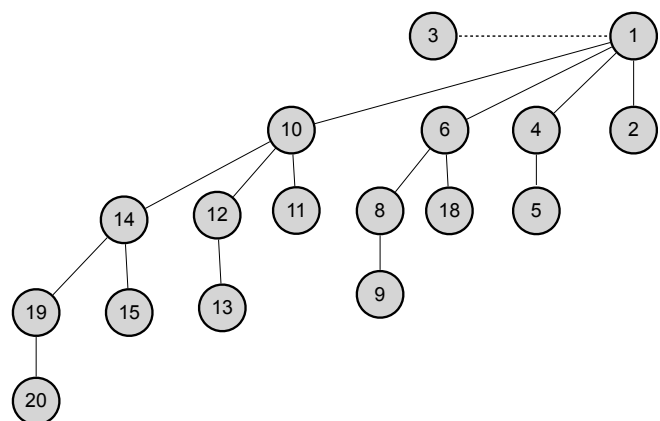
Solution:

Delete 1: [8, 10, 16, 19, 18, 17, 20] → [10, 18, 16, 19, 20, 17]

Delete 2: [10, 18, 16, 19, 20, 17] → [16, 18, 17, 19, 20]

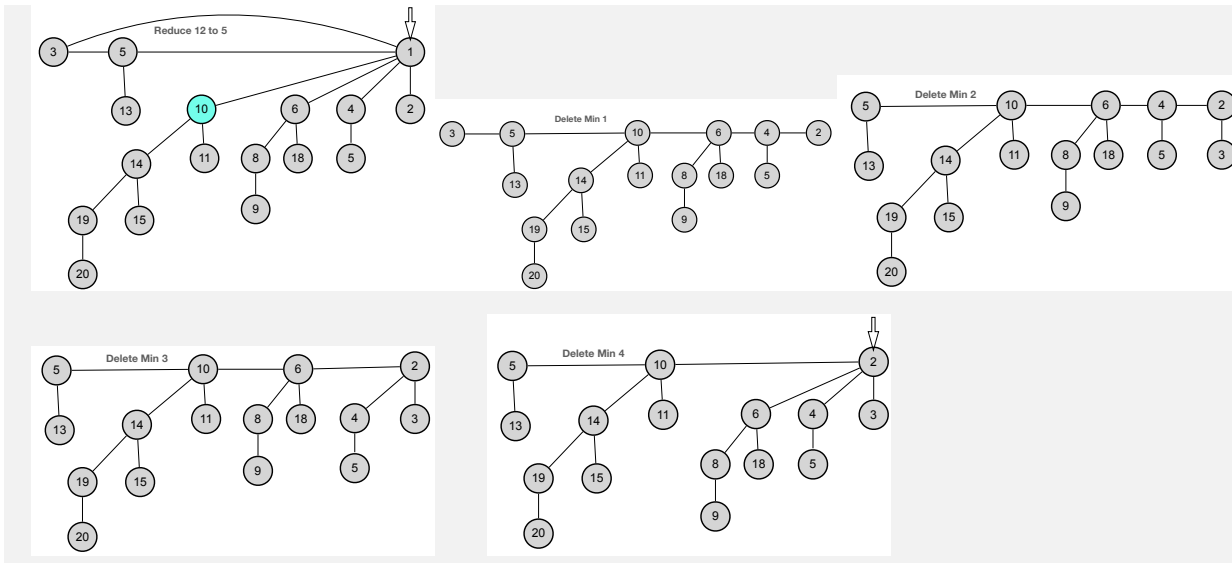
Problem 7 (6 pts)

Consider the Fibonacci min-heap illustrated on the right. Show the Fibonacci heap that results after the following operation: Reduce key 12 to 5. Then, show the Fibonacci heap that results when you delete the minimum in the resulting heap from the first part. Note that you may get different results depending on the merge order you choose. Any reasonable merge order will be fine, so don't worry about it. We don't grade this using a computer program...



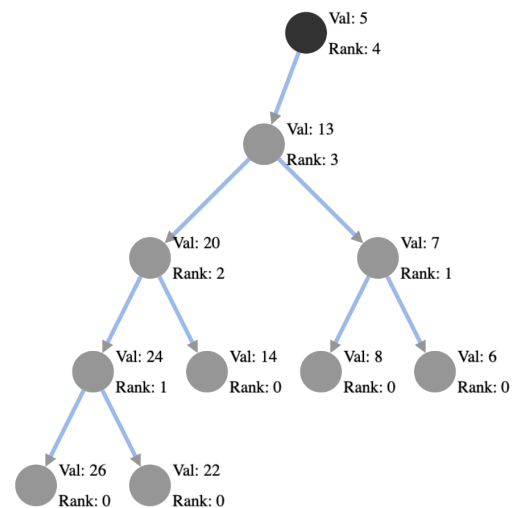
Solution:

The solutions are illustrated below. Decreasing the key from 12 to 5, and then deleting the minimum



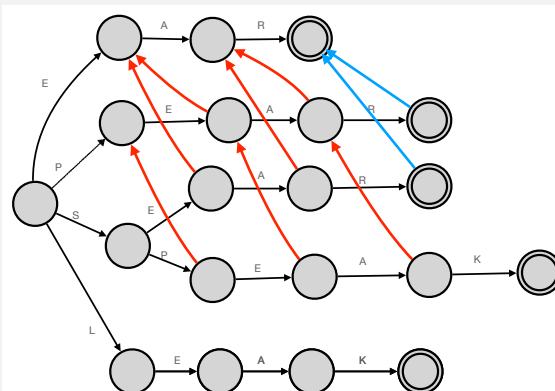
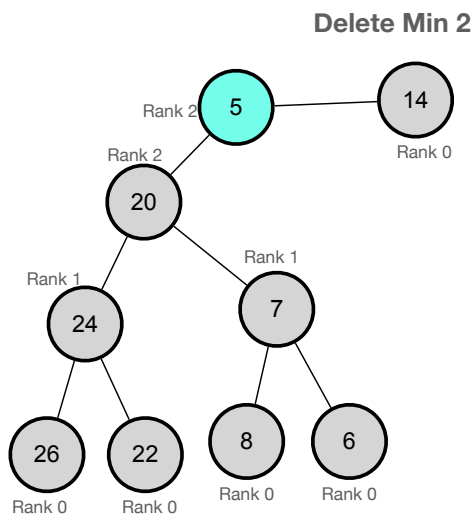
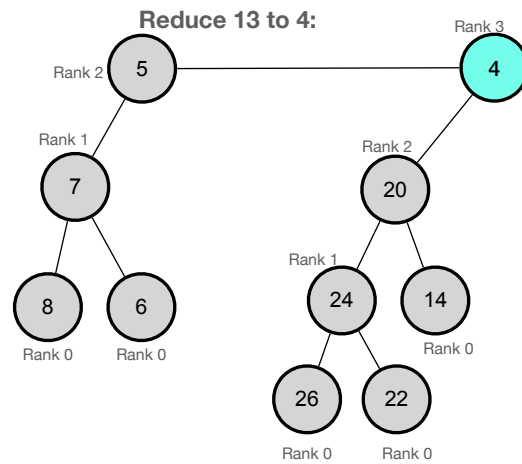
Problem 8 (6 pts)

Consider the rank-pairing min-heap illustrated on the right. Show the rank-pairing heap, including ranks, that results after the following operation: **Reduce key 13 to 4**. Then, show the rank-pairing heap that results when you **delete minimum** in the resulting heap from the first part, using recursive merging, showing the resulting ranks. Note that you may get different results depending on the merge order you choose. Any reasonable merge order will be fine, so don't worry about it.



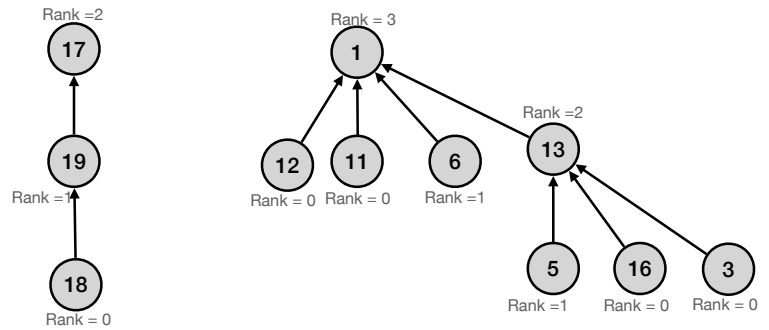
Solution:

The solutions are illustrated below. Decreasing the key from 13 to 4, and then deleting the minimum.



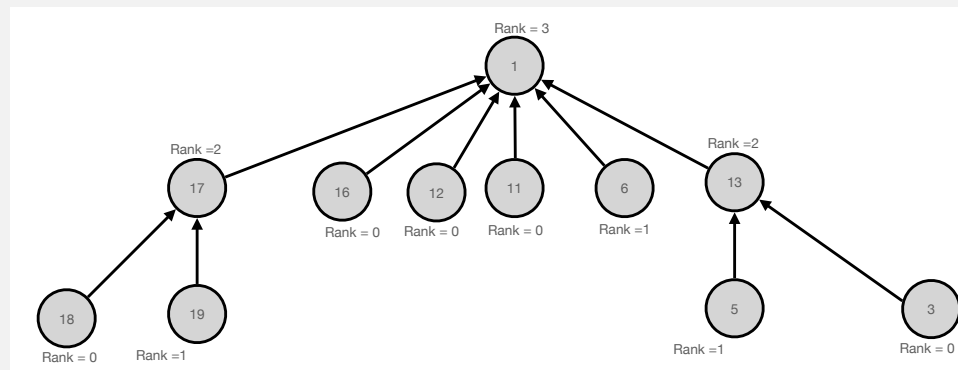
Problem 10 (6 pts)

Consider the two trees, shown on the right, that are part of a disjoint set forest. Suppose we find a relation between keys 16 and 18. Show the disjoint set tree that results from the union operation using merge by rank and path compression, where, **if two roots have the same rank, merge the larger value root under the smaller value root.**



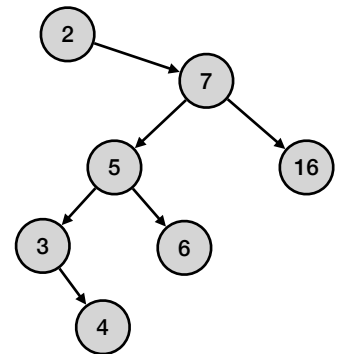
Solution:

The merged trees are shown below, using merge-by-rank and path compression.

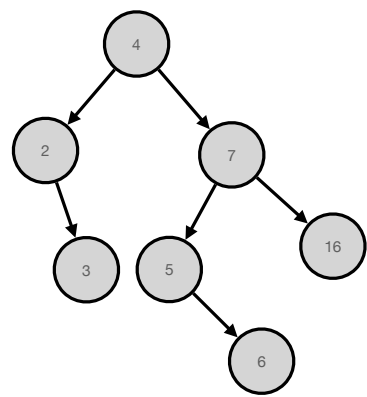


Problem 11 (6 pts)

Consider the splay tree shown on the right. Show the splay tree that results when you find the key 4 (and splay it to the top using bottom up splaying.)

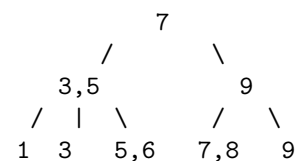


Solution:



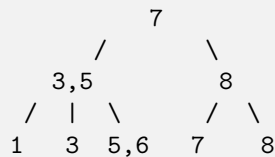
Problem 12 (6 pts)

Consider the B+ tree of order 3 (so min number of children is 2, max is 3), shown on the right. Show the two B+ trees that result when you delete 9 first, then when you delete 8 subsequently.

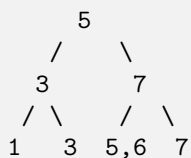


Solution:

Delete 9: borrow from sibling



Del 8 leads to underflow -- must fix.



Problem 13 (6 pts)

For each of these recursions, please give the tightest upper bound for the recursion. You can write your answer as $T(n) \in O(g(n))$ for your best choice of function $g(n)$.

(a) $T(n) = 8T(n/2) + n^3$

Solution:

$\log_2(8) = 3$, so the natural growth is n^3 , which is also the order of the forcing function. By the Master theorem, $T(n) \in \Theta(n^3 \log(n)) \in O(n^3 \log(n))$.

(b) $T(n) = 3T(n/5) + \log^2(n)$.

Solution:

The natural growth is $n^{\log_5(3)}$ is much larger than the forcing function $\log^2(n)$, so the answer is $T(n) \in$

$$O(n^{\log_5(3)})$$

(c) $T(n) = T(n-1) + 2^n$.

Solution:

Note that $T(n) = \sum_{k=1}^n 2^k$. This is like the integral of an exponential, so $T(n) \in O(2^{n+1})$. Note that you can actually sum this, as $2T(n) - T(n) = 2^{n+1} - 2$, thus

$$\sum_{k=1}^n 2^k = 2^{n+1} - 2 \in \Theta(2^{n+1})$$

Problem 14 (6 pts)

Consider a data structure using a sequence of arrays $A_0, A_1, A_2, \dots$ where the i -th array has capacity for 2^i elements. New elements are inserted first into A_0 , and if A_0 is full, its contents are first moved onto A_1 before inserting into A_0 . If A_1 is full, the its contents are first moved onto A_2 . In general, if A_i is full, and we try to move elements into it, we first move the contents of A_i into A_{i+1} to make room to add elements before we add elements to A_i .

- (a) Assume initially you have an empty data structure. You now insert 128 elements. What is the index of the largest array that will contain elements?

Solution:

The maximum number of elements in arrays 0, 1, 2, 3, 4, 5 and 6 is $1 + 2 + 4 + 8 + 16 + 32 + 64 = 127$. Hence, 128 will require elements in A_7 , so it will require 8 arrays. The index of the largest array with 128 elements is 7.

- (b) What is the amortized cost of an individual insertion assuming the data structure starts empty, as the number of insertions n grows? Hint: you can do this by looking at the maximum number of data movements times the number of entries that move that much.

Solution:

For any $n = 2^m$, roughly $n/2$ of the elements will be in the array $A_{\log n + 1}$. Each of these elements will have moved $\log n + 1$ times. Hence, n inserts will require at least $n(\log n + 1)/2$ data movements, so n inserts is in $\Omega(n \log n)$. Since the maximum amount of movement by any element is $\log n + 1$, the index of the largest array with elements, then the maximum amount of work for n elements is $O(n \log n)$. Hence, the amortized cost per insert is $\Theta(\log n)$.

- (c) Suppose each array A_i holds 2 elements only. What is the amortized cost of an insertion now for n insertions?

Solution:

This is the same as inserting at the beginning of an array, with everyone else shifting. The amortized cost per insert is $\Theta(n)$ for n inserts. Again, the total number of inserts is:

$$\sum_{k=1}^n n \in \Theta(n^2)$$

.